Front-End Angular mini project

1. Title Page

- **Project Title**: *"Task Management and Productivity Tracker"*
- **Submitted By**:
  - *Umeshwar Kumar, Tanisha Chhetri, Mervin A, Alen Saijo*
  - *2462166, 2462160, 2462113, 24621*
  - *Umeshwar.kumar@btech.christuniversity.in, tanisha.chhetri@btech.christuniversity.in, mervin.a@btech.christuniversity.in, alen.saijo@btech.christuniversity.in*
- **Course**: *Front-End Development with Angular*
- **Instructor Name**: *Thirupathy and Pallavi*
- **Institution**: *Christ University*
- **Date of Submission**: *23/01/2026*

2. Abstract

This is a **Task Management & Productivity Tracker** built with Angular 17 and Angular Material. The application provides comprehensive CRUD operations with reactive form validation, smart filtering by name, category, status, and priority through a custom pipe, and visual indicators for high-priority and overdue tasks. It features a mock authentication system with route guards protecting edit routes, dark/light theme toggle, and automatic local storage persistence with seed data fallback. An HTTP error interceptor handles errors gracefully with snack bar notifications. The project demonstrates Angular best practices including modular architecture, reactive forms, RxJS state management, custom pipes and directives, and responsive Material UI design.

3. Objectives

a) Provide users with a comprehensive task management application for organizing and tracking daily productivity

b) Implement full CRUD functionality with an intuitive interface for creating, viewing, updating, and deleting tasks

c) Enable efficient task filtering and search capabilities by multiple criteria (name, category, status, priority)

d) Deliver a visually enhanced user experience with priority and overdue task highlighting

e) Ensure secure access to sensitive operations through authentication guards and role-based route protection

Sensitivity: LNT Construction Internal Use

f) Offer flexible theming options with dark and light modes for user preference and accessibility
g) Maintain data persistence using local storage to ensure tasks are retained across sessions
h) Demonstrate modern Angular architecture best practices including reactive programming, modular design, and component-based development
i) Provide robust error handling and user feedback through interceptors and notification systems
j) Build a responsive, user-friendly interface using Angular Material design components

4. Scope of the Project

This project includes a full CRUD task management system with reactive form validation, advanced filtering and search capabilities by name, category, status, and priority, priority highlighting and overdue task indicators, authentication guards protecting edit and delete routes, dark/light theme toggle with Angular Material, local storage persistence with seed data fallback, HTTP error interception with snack bar notifications, responsive navbar and task detail views, and confirmation dialogs for destructive operations. The project excludes backend API or database integration, real authentication systems or multi-user support, task assignment or team collaboration features, calendar or Gantt chart views, email notifications or reminders, mobile applications, and cross-device synchronization.

5. Tools & Technologies Used

The project is built using Angular 17 and TypeScript 5, leveraging RxJS for reactive data handling and asynchronous operations. The user interface is developed with Angular Material components and styled using SCSS to provide a modern, responsive design based on Material Design principles. Data management is handled through Angular Services and the browser's Local Storage API, with initial seed data provided via a JSON file in the assets folder. Core Angular functionality includes Reactive Forms for robust user input, Route Guards for securing application areas, and custom implementations such as the TaskFilterPipe for real-time data transformation and the PriorityHighlight directive for dynamic styling. Additionally, the application utilizes HTTP Interceptors for centralized error handling and Angular Router for seamless navigation. The development lifecycle is supported by the Angular CLI and npm, with quality assurance maintained through the Karma test runner and Jasmine testing framework.

6. Angular Project Structure Overview

```
task-management-app/
├── src/
│   ├── app/
│   │   ├── components/
│   │   │   ├── confirm-dialog/
│   │   │   ├── navbar/
│   │   │   ├── task-detail/
│   │   │   ├── task-form/
│   │   │   └── task-list/
│   │   ├── directives/
│   │   │   └── priority-highlight.directive.ts
│   │   ├── guards/
│   │   │   └── auth.guard.ts
│   │   ├── interceptors/
│   │   │   └── error.interceptor.ts
│   │   ├── models/
│   │   │   └── task.model.ts
│   │   ├── pipes/
│   │   │   └── task-filter.pipe.ts
│   │   ├── services/
│   │   │   ├── auth.service.ts
│   │   │   └── task.service.ts
│   │   ├── shared/
│   │   │   └── material.module.ts
│   │   ├── app.component.ts
│   │   ├── app.component.html
│   │   ├── app-routing.module.ts
│   │   └── app.module.ts
│   ├── assets/
│   │   └── tasks.json
│   └── index.html
```

Sensitivity: LNT Construction Internal Use

7. TypeScript Models and Interfaces

- Task Interface:

```
export interface Task {
        id: string;
        title: string;
        description: string;
        categoryId: string;
        dueDate: string;
        priority: TaskPriority; // 'low' | 'medium' | 'high'
        status: TaskStatus;     // 'pending' | 'in-progress' | 'completed'
        progress: number;
        assignee?: User;
        createdAt: string;
        updatedAt: string;
        tags?: string[];
}
```

- Category Interface:

```
export interface Category {
        id: string;
        name: string;
        color?: string;
}
```

- User Interface:

```
export interface User {
        id: string;
        name: string;
        avatarUrl?: string;
        role?: string;
}
```

8. Angular Features Implemented :

Components & Data Binding:

The application is built using five core components—Navbar, TaskList, TaskForm, TaskDetail, and ConfirmDialog—each structured with isolated templates and SCSS stylesheets for modularity. Dynamic data rendering is achieved through extensive use of property binding to sync task properties with the UI, while event binding facilitates user interactions such as adding new tasks, performing edits, deleting items, and toggling completion statuses.

Directives:

Template logic is handled by built-in structural directives like *ngFor for iterating through task and category collections and *ngIf for conditional element rendering based on application state. The application also utilizes attribute directives like [ngClass] for dynamic styling (e.g., category color coding) and features a custom appPriorityHighlight directive that provides specialized visual emphasis for high-priority or overdue tasks.

Services:

The business logic is encapsulated within singleton services: the TaskService handles data management, including fetching seed data from JSON and synchronizing changes with the browser's local storage, while the AuthService manages a mock authentication flow to track the current user's session and permissions.

Routing:

The Angular Router is configured to provide a seamless navigation experience across different views, including the dashboard list, specific task details, and the task management forms. Navigation is implemented via both routerLink directives and programmatic navigation, with AuthGuard protecting sensitive routes to ensure that only "authenticated" users can create or modify tasks.

Forms:

Task input is managed using Reactive Forms, providing high-level control over form validation and state. The TaskForm utilizes FormBuilder to create complex validation rules for mandatory fields, date logic, and progress tracking, while also integrating Material components like dropdowns for category selection and priority levels to ensure a structured and error-free data entry process.

Sensitivity: LNT Construction Internal Use

9. CSS Styling Strategy

- Used component-specific SCSS files for scoped and modular styling
- Applied a modern Material Design approach using Angular Material components
- Leveraged Material cards and elevations for distinct element visibility
- Implemented a dynamic theme system with dark and light mode support
- Used primary and accent color palettes for consistent branding
- Integrated flexbox and CSS grid for structured layout
- Focused on high readability with proper typography and whitespace
- Utilized Angular Material's built-in theming engine for global styles
- Ensured responsive design using SCSS media queries and Material's layout system
- Added custom highlight styles via directives for priority and status visualization

10. Key Features Implemented

- Task Management:

  The application provides a centralized hub for tracking productivity, allowing users to view and manage all active and completed tasks within a responsive container. Each task entry displays essential details such as titles, priority-based color coding, full descriptions, and due dates. To ensure data safety, a robust deletion workflow is implemented, requiring users to confirm their intent through a dialog before a task is permanently removed.
- Course Catalog:

  To handle large volumes of tasks, the dashboard includes a sophisticated real-time filtering system powered by a custom Angular pipe. Users can instantly narrow down their view by searching for specific task names or by selecting criteria like category, priority level, and current status. The interface provides immediate feedback by dynamically updating the task count and layout based on the active filters, ensuring a clear and organized overview at all times.
- Enrollment System:

  Creating and modifying tasks is facilitated through a robust Reactive Forms implementation that ensures data integrity through multi-step validation. The TaskForm guides users through entering required information, selecting from predefined categories and priority levels, and assigning due dates. All changes are automatically synchronized with the browser's local storage, ensuring that user data persists even after refreshing the page or closing the browser.
- Navigation:

  The user experience is enhanced by a fixed, responsive navigation bar that provides consistent access to key application areas while highlighting the active route for easy orientation. The interface supports a dynamic theme engine, allowing users to switch between light and dark modes according to their preference. Seamless interaction feedback is provided throughout the app via Material snack bars for notifications and smooth transitions between the task list and detailed views.

Sensitivity: LNT Construction Internal Use

11. Challenges Faced and Solutions

Challenge 1: Managing application state across multiple independent components.
Solution: Implemented a centralized `TaskService` using RxJS Observables to ensure a single source of truth for task data.

Challenge 2: Persisting data without a backend database.
Solution: Developed a synchronization layer between the `TaskService` and the browser's `localStorage` API to ensure data retention across sessions.

Challenge 3: Implementing real-time filtering for complex task objects.
Solution: Created a custom `TaskFilterPipe` that handles multiple criteria including title, category, status, and priority simultaneously.

Challenge 4: Ensuring consistent UI across various screen sizes.
Solution: Utilized Angular Material's responsive layout system combined with custom SCSS media queries for fluid component scaling.

12. Learning Outcomes

Through the development of this project, I gained significant practical experience in setting up and configuring modern Angular 17 projects using the Angular CLI and architecting modular application structures. I deepened my understanding of TypeScript by defining rigorous interfaces for type safety and mastered the use of Angular Services and RxJS for state management. The project provided an opportunity to create custom directives and pipes to extend template functionality, while also implementing secure navigation through Angular Router and Route Guards. Additionally, I improved my skills in designing polished user interfaces with Angular Material and SCSS, as well as managing complex user input through Reactive Forms.

13. Future Enhancements

The project has several avenues for future growth, including the integration of a real backend API and database for centralized storage and the implementation of a robust authentication system using Firebase or JWT. User interaction could be improved by adding drag-and-drop functionality for task reordering and status updates, along with a calendar view for better deadline visualization. Further functional additions could include support for subtask dependencies, file attachments, and an automated notification system for overdue items. Finally, developing a productivity dashboard with charts and analytics would provide users with deeper insights into their task management trends.

Sensitivity: LNT Construction Internal Use

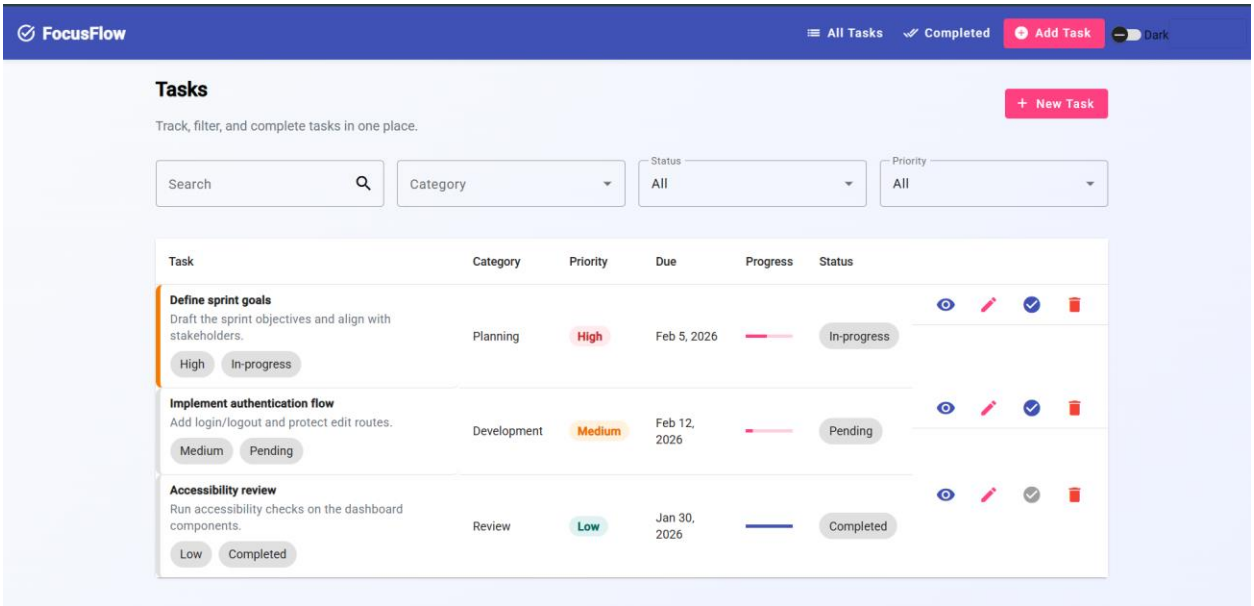
14. Conclusion

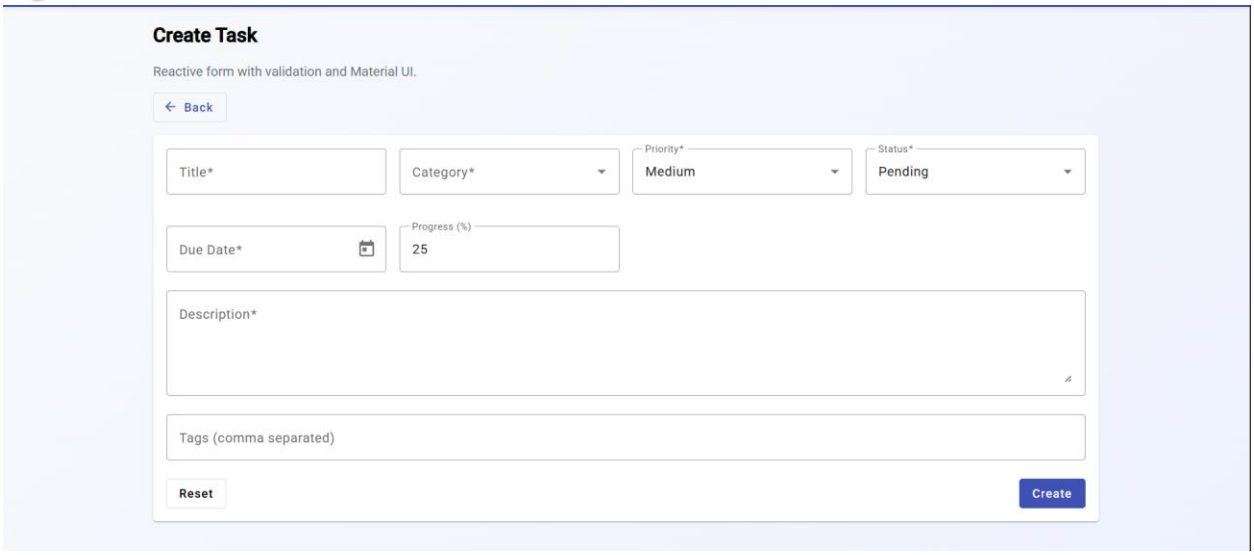
The Task Management & Productivity Tracker successfully demonstrates the power and flexibility of Angular 17 in building modern, responsive web applications. By integrating core Angular features like Reactive Forms, custom pipes, and route guards with a polished Angular Material UI, the project provides a seamless user experience for managing daily tasks. While currently utilizing local storage for persistence, the application's modular architecture is designed for easy scalability and future integration with a dedicated backend. Overall, this project serves as a robust foundation for a productivity tool, showcasing best practices in component-based design, state management, and type-safe development with TypeScript.

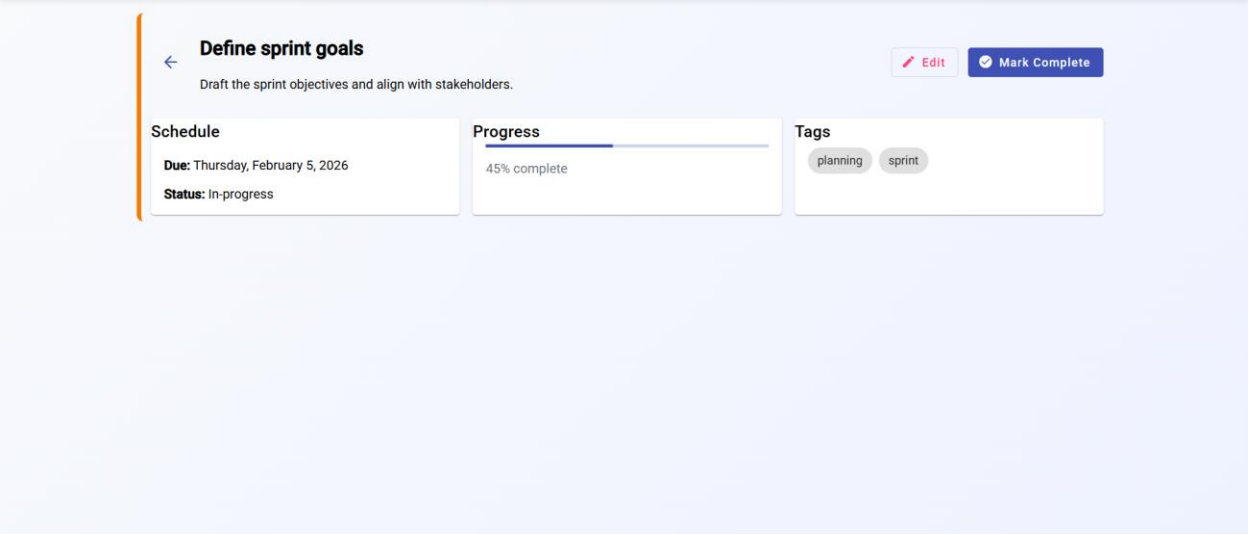
15. References

Angular Official Documentation - https://angular.io/docs
Angular Material Documentation - https://material.angular.io/
TypeScript Handbook - https://www.typescriptlang.org/docs/
Angular CLI Documentation - https://angular.io/cli
MDN Web Docs - https://developer.mozilla.org/

16. Screenshots







Appendix

Installation Steps:

bashnpm install

ng serve

Access the application:

Navigate to http://localhost:4200